



```
In [1]: import pandas as pd
df = pd.read_excel('/content/text_similarity_dataset.xlsx')
display(df.head())
```

	doc_id	topic	text
0	1	sports	The football team won the match after scoring ...
1	2	sports	Cricket players trained hard to improve their ...
2	3	sports	The tennis championship attracted athletes fro...
3	4	sports	The coach praised the players for their teamwo...
4	5	sports	The basketball match was intense and went into...

```
In [21]: import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
import string
import re
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt_tab')
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words('english'))
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
In [28]: def to_lowercase(text):
        return text.lower()

def remove_punctuation_numbers(text):
    text = text.translate(str.maketrans('', '', string.punctuation))
    text = re.sub(r'\d+', '', text)
    return text

def remove_stopwords_func(tokens):
    return [word for word in tokens if word not in stop_words]

def tokenize_text(text):
    return word_tokenize(text)

def lemmatize_tokens(tokens):
    return [lemmatizer.lemmatize(word) for word in tokens]
```

```
In [6]: def preprocess_text(text, apply_lemmatization=True):
        text = to_lowercase(text)
```

```

text = remove_punctuation_numbers(text)
tokens = tokenize_text(text)
tokens = remove_stopwords_func(tokens)
if apply_lemmatization:
    tokens = lemmatize_tokens(tokens)
return ' '.join(tokens)

df['preprocessed_text'] = df['text'].apply(preprocess_text)
display(df[['text', 'preprocessed_text']].head())

```

	text	preprocessed_text
0	The football team won the match after scoring ...	football team match scoring two goal final minute
1	Cricket players trained hard to improve their ...	cricket player trained hard improve batting bo...
2	The tennis championship attracted athletes fro...	tennis championship attracted athlete many cou...
3	The coach praised the players for their teamwo...	coach praised player teamwork discipline
4	The basketball match was intense and went into...	basketball match intense went overtime

In [7]:

```

from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(df['preprocessed_text'])
print('Shape of TF-IDF matrix:', tfidf_matrix.shape)

```

Shape of TF-IDF matrix: (24, 130)

In [23]:

```

from sklearn.metrics.pairwise import cosine_similarity
cosine_sim_matrix = cosine_similarity(tfidf_matrix)

cosine_sim_df = pd.DataFrame(cosine_sim_matrix, index=df.index, columns=df.index)

print("Cosine Similarity Matrix:")
display(cosine_sim_df)

print("\nInterpretation of 5 sample pairs:")

sim_0_1 = cosine_sim_df.loc[0, 1]
print(f"\nDocument 0 and Document 1 (Sports related): Similarity = {sim_0_1:.4f}")
print(f" Document 0: {df['text'][0]}")
print(f" Document 1: {df['text'][1]}")

sim_0_15 = cosine_sim_df.loc[0, 15]
print(f"\nDocument 0 (Sports) and Document 15 (Business): Similarity = {sim_0_15:.4f}")
print(f" Document 0: {df['text'][0]}")
print(f" Document 15: {df['text'][15]}")

```

```

sim_5_6 = cosine_sim_df.loc[5, 6]
print(f"\nDocument 5 and Document 6 (Politics related): Similarity = {sim_5_6}")
print(f"  Document 5: {df['text'][5]}")
print(f"  Document 6: {df['text'][6]}")

sim_10_11 = cosine_sim_df.loc[10, 11]
print(f"\nDocument 10 and Document 11 (Technology related): Similarity = {sim_10_11}")
print(f"  Document 10: {df['text'][10]}")
print(f"  Document 11: {df['text'][11]}")

sim_10_20 = cosine_sim_df.loc[10, 20]
print(f"\nDocument 10 (Technology) and Document 20 (Health): Similarity = {sim_10_20}")
print(f"  Document 10: {df['text'][10]}")
print(f"  Document 20: {df['text'][20]}")

```

Cosine Similarity Matrix:

	0	1	2	3	4	5	6	7
0	1.000000	0.000000	0.000000	0.00000	0.128364	0.000000	0.000000	0.000000
1	0.000000	1.000000	0.000000	0.13019	0.000000	0.000000	0.000000	0.000000
2	0.000000	0.000000	1.000000	0.00000	0.000000	0.000000	0.000000	0.151787
3	0.000000	0.130190	0.000000	1.00000	0.000000	0.000000	0.000000	0.000000
4	0.128364	0.000000	0.000000	0.00000	1.000000	0.000000	0.000000	0.000000
5	0.000000	0.000000	0.000000	0.00000	0.000000	1.000000	0.000000	0.000000
6	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	1.000000	0.000000
7	0.000000	0.000000	0.151787	0.00000	0.000000	0.000000	0.000000	1.000000
8	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
9	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
10	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.127097	0.000000
11	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
12	0.000000	0.000000	0.000000	0.00000	0.000000	0.383125	0.000000	0.000000
13	0.000000	0.000000	0.129548	0.00000	0.000000	0.123420	0.000000	0.000000
14	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.129548	0.000000
15	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
16	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
17	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
18	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
19	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
20	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
21	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
22	0.000000	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000
23	0.000000	0.133245	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000

24 rows × 24 columns

Interpretation of 5 sample pairs:

Document 0 and Document 1 (Sports related): Similarity = 0.0000

Document 0: The football team won the match after scoring two goals in the final minutes.

Document 1: Cricket players trained hard to improve their batting and bowling skills.

Document 0 (Sports) and Document 15 (Business): Similarity = 0.0000

Document 0: The football team won the match after scoring two goals in the final minutes.

Document 15: Mental health awareness is important for overall well-being.

Document 5 and Document 6 (Politics related): Similarity = 0.0000

Document 5: Regular exercise and sports help maintain physical fitness.

Document 6: The government announced new economic policies to control inflation.

Document 10 and Document 11 (Technology related): Similarity = 0.0000

Document 10: Political leaders debated healthcare and employment policies.

Document 11: Democracy allows citizens to vote and express their opinions.

Document 10 (Technology) and Document 20 (Health): Similarity = 0.0000

Document 10: Political leaders debated healthcare and employment policies.

Document 20: Cybersecurity is essential to protect user data online.

```
In [22]: from sklearn.metrics.pairwise import cosine_similarity
cosine_sim_matrix = cosine_similarity(tfidf_matrix)
cosine_sim_df = pd.DataFrame(cosine_sim_matrix, index=df.index, columns=df.index)
print("Cosine Similarity Matrix:")
display(cosine_sim_df)
print("\nInterpretation of 5 sample pairs:")

sim_5_12 = cosine_sim_df.loc[5, 12]
print(f"\nDocument 5 and Document 12 (Exercise/Fitness related): Similarity = {sim_5_12}")
print(f"Document 5: {df['text'][5]}")
print(f"Document 12: {df['text'][12]}")

sim_2_7 = cosine_sim_df.loc[2, 7]
print(f"\nDocument 2 and Document 7 (Athletes/Sports related): Similarity = {sim_2_7}")
print(f"Document 2: {df['text'][2]}")
print(f"Document 7: {df['text'][7]}")

sim_1_3 = cosine_sim_df.loc[1, 3]
print(f"\nDocument 1 and Document 3 (Sports terms related): Similarity = {sim_1_3}")
print(f"Document 1: {df['text'][1]}")
print(f"Document 3: {df['text'][3]}")

sim_6_10 = cosine_sim_df.loc[6, 10]
print(f"\nDocument 6 and Document 10 (Politics/Policy related): Similarity = {sim_6_10}")
print(f"Document 6: {df['text'][6]}")
print(f"Document 10: {df['text'][10]}")

sim_0_15 = cosine_sim_df.loc[0, 15]
print(f"\nDocument 0 (Sports) and Document 15 (Health): Similarity = {sim_0_15}")
```

```
print(f" Document 0: {df['text'][0]}")
print(f" Document 15: {df['text'][15]}")
```

Cosine Similarity Matrix:

	0	1	2	3	4	5	6	7
0	1.000000	0.000000	0.000000	0.000000	0.128364	0.000000	0.000000	0.000000
1	0.000000	1.000000	0.000000	0.13019	0.000000	0.000000	0.000000	0.000000
2	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.151787
3	0.000000	0.130190	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000
4	0.128364	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000
5	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000
6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000
7	0.000000	0.000000	0.151787	0.000000	0.000000	0.000000	0.000000	1.000000
8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
9	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
10	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.127097	0.000000
11	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
12	0.000000	0.000000	0.000000	0.000000	0.000000	0.383125	0.000000	0.000000
13	0.000000	0.000000	0.129548	0.000000	0.000000	0.123420	0.000000	0.000000
14	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.129548	0.000000
15	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
16	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
17	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
18	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
19	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
20	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
21	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
22	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
23	0.000000	0.133245	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

24 rows × 24 columns

Interpretation of 5 sample pairs:

Document 5 and Document 12 (Exercise/Fitness related): Similarity = 0.3831

Document 5: Regular exercise and sports help maintain physical fitness.

Document 12: Doctors recommend regular exercise to maintain good health.

Document 2 and Document 7 (Athletes/Sports related): Similarity = 0.1518

Document 2: The tennis championship attracted athletes from many countries.

Document 7: Elections were conducted peacefully across the country.

Document 1 and Document 3 (Sports terms related): Similarity = 0.1302

Document 1: Cricket players trained hard to improve their batting and bowling skills.

Document 3: The coach praised the players for their teamwork and discipline.

Document 6 and Document 10 (Politics/Policy related): Similarity = 0.1271

Document 6: The government announced new economic policies to control inflation.

Document 10: Political leaders debated healthcare and employment policies.

Document 0 (Sports) and Document 15 (Health): Similarity = 0.0000

Document 0: The football team won the match after scoring two goals in the final minutes.

Document 15: Mental health awareness is important for overall well-being.

```
In [10]: def jaccard_similarity(set1, set2):
          intersection = len(set1.intersection(set2))
          union = len(set1.union(set2))
          if union == 0:
              return 0.0
          return intersection / union

          print("Defined jaccard_similarity function.")
```

Defined jaccard_similarity function.

```
In [11]: df['tokens'] = df['preprocessed_text'].apply(lambda x: set(x.split()))
          print("Created 'tokens' column with sets of preprocessed words.")
```

Created 'tokens' column with sets of preprocessed words.

```
In [12]: import numpy as np

          num_documents = len(df)
          jaccard_sim_matrix = np.zeros((num_documents, num_documents))
          print(f"Initialized a {num_documents}x{num_documents} matrix for Jaccard simil
```

Initialized a 24x24 matrix for Jaccard similarity scores.

```
In [13]: for i in range(num_documents):
          for j in range(num_documents):
              set1 = df['tokens'][i]
              set2 = df['tokens'][j]
              jaccard_sim_matrix[i, j] = jaccard_similarity(set1, set2)
          print("Jaccard similarity matrix computed.")
```

Jaccard similarity matrix computed.

```
In [14]: jaccard_sim_df = pd.DataFrame(jaccard_sim_matrix, index=df.index, columns=df.index)
print("Jaccard Similarity Matrix:")
display(jaccard_sim_df)
```

Jaccard Similarity Matrix:

	0	1	2	3	4	5	6	7	8
0	1.000000	0.000000	0.000000	0.000000	0.083333	0.000000	0.000000	0.0	0.0
1	0.000000	1.000000	0.000000	0.083333	0.000000	0.000000	0.000000	0.0	0.0
2	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.1	0.0
3	0.000000	0.083333	0.000000	1.000000	0.000000	0.000000	0.000000	0.0	0.0
4	0.083333	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.0	0.0
5	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.0	0.0
6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.0	0.0
7	0.000000	0.000000	0.100000	0.000000	0.000000	0.000000	0.000000	1.0	0.0
8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	1.0
9	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
10	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.083333	0.0	0.0
11	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
12	0.000000	0.000000	0.000000	0.000000	0.000000	0.272727	0.000000	0.0	0.0
13	0.000000	0.000000	0.083333	0.000000	0.000000	0.076923	0.000000	0.0	0.0
14	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.083333	0.0	0.0
15	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
16	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
17	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
18	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
19	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
20	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
21	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
22	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0
23	0.000000	0.083333	0.000000	0.000000	0.000000	0.000000	0.000000	0.0	0.0

24 rows × 24 columns

```
In [24]: print("\nInterpretation of 5 sample pairs with Jaccard and Cosine Similarity:")
idx1, idx2 = 5, 12
j_sim1 = jaccard_sim_df.loc[idx1, idx2]
```



```

c_sim1 = cosine_sim_df.loc[idx1, idx2]
print(f"\nDocument {idx1} and Document {idx2} (Exercise/Fitness related):\n J")
print(f" Document {idx1}: {df['text'][idx1]}\n Document {idx2}: {df['text']["

idx1, idx2 = 2, 7
j_sim2 = jaccard_sim_df.loc[idx1, idx2]
c_sim2 = cosine_sim_df.loc[idx1, idx2]
print(f"\nDocument {idx1} and Document {idx2}:\n Jaccard Similarity = {j_sim2}")
print(f" Document {idx1}: {df['text'][idx1]}\n Document {idx2}: {df['text']["

idx1, idx2 = 1, 3
j_sim3 = jaccard_sim_df.loc[idx1, idx2]
c_sim3 = cosine_sim_df.loc[idx1, idx2]
print(f"\nDocument {idx1} and Document {idx2} (Sports terms related):\n Jaccard")
print(f" Document {idx1}: {df['text'][idx1]}\n Document {idx2}: {df['text']["

idx1, idx2 = 6, 10
j_sim4 = jaccard_sim_df.loc[idx1, idx2]
c_sim4 = cosine_sim_df.loc[idx1, idx2]
print(f"\nDocument {idx1} and Document {idx2} (Politics/Policy related):\n Jaccard")
print(f" Document {idx1}: {df['text'][idx1]}\n Document {idx2}: {df['text']["

idx1, idx2 = 0, 15
j_sim5 = jaccard_sim_df.loc[idx1, idx2]
c_sim5 = cosine_sim_df.loc[idx1, idx2]
print(f"\nDocument {idx1} (Sports) and Document {idx2} (Health):\n Jaccard Similarity")
print(f" Document {idx1}: {df['text'][idx1]}\n Document {idx2}: {df['text']["

```

Interpretation of 5 sample pairs with Jaccard and Cosine Similarity:

Document 5 and Document 12 (Exercise/Fitness related):

Jaccard Similarity = 0.2727

Cosine Similarity = 0.3831

Document 5: Regular exercise and sports help maintain physical fitness.

Document 12: Doctors recommend regular exercise to maintain good health.

Document 2 and Document 7:

Jaccard Similarity = 0.1000

Cosine Similarity = 0.1518

Document 2: The tennis championship attracted athletes from many countries.

Document 7: Elections were conducted peacefully across the country.

Document 1 and Document 3 (Sports terms related):

Jaccard Similarity = 0.0833

Cosine Similarity = 0.1302

Document 1: Cricket players trained hard to improve their batting and bowling skills.

Document 3: The coach praised the players for their teamwork and discipline.

Document 6 and Document 10 (Politics/Policy related):

Jaccard Similarity = 0.0833

Cosine Similarity = 0.1271

Document 6: The government announced new economic policies to control inflation.

Document 10: Political leaders debated healthcare and employment policies.

Document 0 (Sports) and Document 15 (Health):

Jaccard Similarity = 0.0000

Cosine Similarity = 0.0000

Document 0: The football team won the match after scoring two goals in the final minutes.

Document 15: Mental health awareness is important for overall well-being.

```
In [16]: import nltk
from nltk.corpus import wordnet
from nltk import pos_tag, word_tokenize

try:
    nltk.data.find('taggers/averaged_perceptron_tagger.zip')
except LookupError:
    nltk.download('averaged_perceptron_tagger')

print("NLTK modules imported and 'averaged_perceptron_tagger' checked/downloaded")
```

NLTK modules imported and 'averaged_perceptron_tagger' checked/downloaded.

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.
```

```
In [17]: def get_wordnet_pos(tag):
        if tag.startswith('J'):
            return wordnet.ADJ
```

```

elif tag.startswith('V'):
    return wordnet.VERB
elif tag.startswith('N'):
    return wordnet.NOUN
elif tag.startswith('R'):
    return wordnet.ADV
else:
    return wordnet.NOUN
print("Defined get_wordnet_pos helper function.")

```

Defined get_wordnet_pos helper function.

```

In [18]: def compute_word_similarity(word1, word2):
    synsets1 = wordnet.synsets(word1)
    synsets2 = wordnet.synsets(word2)

    if not synsets1 or not synsets2:
        return 0.0

    max_similarity = 0.0
    for ss1 in synsets1:
        for ss2 in synsets2:
            similarity = ss1.wup_similarity(ss2)
            if similarity is not None and similarity > max_similarity:
                max_similarity = similarity
    return max_similarity

    print("Defined compute_word_similarity function.")

```

Defined compute_word_similarity function.

```

In [19]: def compute_sentence_similarity(sentence1, sentence2):
    def get_tokens_for_semantic_sim(text):
        text = to_lowercase(text)
        text = remove_punctuation_numbers(text)
        tokens = tokenize_text(text)
        tokens = remove_stopwords_func(tokens)
        tokens = lemmatize_tokens(tokens)
        return tokens

    tokens1 = get_tokens_for_semantic_sim(sentence1)
    tokens2 = get_tokens_for_semantic_sim(sentence2)

    if not tokens1 or not tokens2:
        return 0.0

    s1_to_s2_scores = []
    for word1 in tokens1:
        max_word_sim = 0.0
        for word2 in tokens2:
            sim = compute_word_similarity(word1, word2)
            if sim > max_word_sim:
                max_word_sim = sim
        s1_to_s2_scores.append(max_word_sim)

```

```

avg_s1_to_s2 = sum(s1_to_s2_scores) / len(s1_to_s2_scores) if s1_to_s2_sco

s2_to_s1_scores = []
for word2 in tokens2:
    max_word_sim = 0.0
    for word1 in tokens1:
        sim = compute_word_similarity(word2, word1)
        if sim > max_word_sim:
            max_word_sim = sim
    s2_to_s1_scores.append(max_word_sim)
avg_s2_to_s1 = sum(s2_to_s1_scores) / len(s2_to_s1_scores) if s2_to_s1_sco

return (avg_s1_to_s2 + avg_s2_to_s1) / 2.0

print("Defined compute_sentence_similarity function.")

```

Defined compute_sentence_similarity function.

```

In [27]: print("\nWordNet Semantic Similarity Analysis (10 Sample Pairs):\n")
sentence_pairs = [(5, 12), (8, 13), (0, 1), (6, 10), (9, 20), (14, 17), (0, 15), (1,

for idx, (i, j) in enumerate(sentence_pairs):
    sent1 = df['text'][i]
    sent2 = df['text'][j]
    semantic_sim = compute_sentence_similarity(sent1, sent2)
    jaccard_sim = jaccard_sim_df.loc[i, j]
    cosine_sim = cosine_sim_df.loc[i, j]

    print(f"--- Pair {idx + 1}: Document {i} and Document {j} ---")
    print(f"  Document {i}: {sent1}")
    print(f"  Document {j}: {sent2}")
    print(f"  WordNet Semantic Similarity = {semantic_sim:.4f}")
    print(f"  Jaccard Similarity           = {jaccard_sim:.4f}")
    print(f"  Cosine Similarity              = {cosine_sim:.4f}")

    if (i, j) == (5, 12):
        print("  Interpretation: High semantic similarity")
        print("  Interpretation: High semantic similarity")
    elif (i, j) == (0, 1):
        print("  Interpretation: Moderate semantic similarity")
    elif (i, j) == (6, 10):
        print("  Interpretation: Moderate semantic similarity")
    elif (i, j) == (9, 20):
        print("  Interpretation: Moderate semantic similarity")
    elif (i, j) == (14, 17):
        print("  Interpretation: Moderate semantic similarity")
    elif (i, j) == (0, 15):
        print("  Interpretation: Low semantic similarity")
    elif (i, j) == (1, 7):
        print("  Interpretation: Low semantic similarity")
    elif (i, j) == (5, 16):
        print("  Interpretation: Low semantic similarity")
    elif (i, j) == (19, 23):
        print("  Interpretation: Low semantic similarity.")

```

```
print("\n")
```

WordNet Semantic Similarity Analysis (10 Sample Pairs):

--- Pair 1: Document 5 and Document 12 ---

Document 5: Regular exercise and sports help maintain physical fitness.
Document 12: Doctors recommend regular exercise to maintain good health.
WordNet Semantic Similarity = 0.8336
Jaccard Similarity = 0.2727
Cosine Similarity = 0.3831
Interpretation: High semantic similarity
Interpretation: High semantic similarity

--- Pair 2: Document 8 and Document 13 ---

Document 8: The prime minister addressed the nation on national security issues.
Document 13: A balanced diet helps prevent many chronic diseases.
WordNet Semantic Similarity = 0.6014
Jaccard Similarity = 0.0000
Cosine Similarity = 0.0000

--- Pair 3: Document 0 and Document 1 ---

Document 0: The football team won the match after scoring two goals in the final minutes.
Document 1: Cricket players trained hard to improve their batting and bowling skills.
WordNet Semantic Similarity = 0.6302
Jaccard Similarity = 0.0000
Cosine Similarity = 0.0000
Interpretation: Moderate semantic similarity

--- Pair 4: Document 6 and Document 10 ---

Document 6: The government announced new economic policies to control inflation.
Document 10: Political leaders debated healthcare and employment policies.
WordNet Semantic Similarity = 0.6927
Jaccard Similarity = 0.0833
Cosine Similarity = 0.1271
Interpretation: Moderate semantic similarity

--- Pair 5: Document 9 and Document 20 ---

Document 9: Parliament passed a bill focusing on education reforms.
Document 20: Cybersecurity is essential to protect user data online.
WordNet Semantic Similarity = 0.4586
Jaccard Similarity = 0.0000
Cosine Similarity = 0.0000
Interpretation: Moderate semantic similarity

--- Pair 6: Document 14 and Document 17 ---

Document 14: The hospital introduced new technology for patient care.
Document 17: Proper sleep improves immunity and concentration.

WordNet Semantic Similarity = 0.5509
Jaccard Similarity = 0.0000
Cosine Similarity = 0.0000
Interpretation: Moderate semantic similarity

--- Pair 7: Document 0 and Document 15 ---

Document 0: The football team won the match after scoring two goals in the final minutes.

Document 15: Mental health awareness is important for overall well-being.

WordNet Semantic Similarity = 0.4743

Jaccard Similarity = 0.0000

Cosine Similarity = 0.0000

Interpretation: Low semantic similarity

--- Pair 8: Document 1 and Document 7 ---

Document 1: Cricket players trained hard to improve their batting and bowling skills.

Document 7: Elections were conducted peacefully across the country.

WordNet Semantic Similarity = 0.5376

Jaccard Similarity = 0.0000

Cosine Similarity = 0.0000

Interpretation: Low semantic similarity

--- Pair 9: Document 19 and Document 23 ---

Document 19: Smartphones have changed the way people communicate.

Document 23: Advances in technology improve efficiency and productivity.

WordNet Semantic Similarity = 0.5404

Jaccard Similarity = 0.0000

Cosine Similarity = 0.0000

Interpretation: Low semantic similarity.